When Does Graph Structure Help in Multi-Agent Reinforcement Learning? A Controlled Empirical Study

Anonymous authors
Paper under double-blind review

Abstract

Graph neural network (GNN) value-factorisation methods are widely claimed to outperform graph-free baselines in cooperative multi-agent reinforcement learning (MARL) by exploiting relational inductive bias. The conditions under which the claim holds are under-characterised: prior work typically reports a single benchmark in a single graph regime, conflating the contribution of the graph with that of added capacity, and leaving the alignment between the GNN's receptive field and the task's coordination structure unexamined. We close this gap with a controlled empirical study on CoordGrid, a cooperative gridworld whose coordination graph is itself the experimental variable. Holding the per-agent encoder, state-conditioned mixer, optimiser, and exploration schedule fixed across algorithms, we test pre-registered hypotheses on a structured graph (H1), against a random graph of matched edge density (H2), and across a GNN-depth $\times$ graph-diameter grid (H3), and we add a parameter-matched no-graph control (MLP-QMIX) that isolates the graph from the extra capacity it brings. Three findings emerge. First, GNN-QMIX *underperforms* the structurally simpler QMIX — opposite to the prevailing claim — and the gap is *identified*, not confounded: against MLP-QMIX, which has byte-identical parameter count but no graph aggregation, GNN-QMIX is significantly worse (Cohen's $d = -3.9$ at $N = 8$; Mann–Whitney $p = 0.0002$), while MLP-QMIX matches QMIX. The penalty is therefore the graph aggregation itself, not added capacity, and it *grows* with team size. The effect survives $5\times$ the training budget (150,000 steps, 10 seeds), ruling out undertraining; and the graph also *destabilises* learning (seed SD 30.9 at $N = 8$, vs. 3.3 for the parameter-matched no-graph control). Second, the depth–diameter alignment heuristic (deeper GNN for larger graph diameter) is *not* confirmed at the family level (1 of 3 diameters pass); instead, the data reveal a sharp cliff above $L = 2$, with $L \geq 3$ catastrophic across every diameter tested. Third, the practitioner rule we recommend is correspondingly blunter than H3 anticipated: do not stack more than two GCN layers in front of a QMIX-style mixer in small-team CTDE settings. We release the harness, configs, and plotting code at the URL in Appendix C.

1 Introduction

Cooperative multi-agent reinforcement learning (MARL) underlies progress on team coordination problems from traffic signal control to multi-robot manipulation. Value-decomposition methods — in which a joint team Q-value Q_{tot} is factorised into per-agent utilities Q_i for centralised training and decentralised execution — have emerged as a workhorse: VDN (Sunehag et al., 2018) factorises Q_{tot} as a sum, QMIX(Rashid et al., 2018; 2020) as a monotonic mixture, and several recent methods extend the mixer with graph neural networks (GNNs) over an externally specified or learned coordination graph (Jiang et al., 2020; Liu et al., 2020; Böhmer et al., 2020; Naderializadeh et al., 2020).

The intuition for graph-based MARL is straightforward: when agents form a coordination *structure*, a model whose architecture mirrors that structure should generalise better and converge faster. This intuition is widely cited as motivation in the literature, yet the conditions under which it actually holds are surprisingly under-tested. Three specific gaps motivate the present study:

1. **Graph regime is rarely varied.** Most empirical comparisons fix one environment with one (often implicit) coordination graph, leaving open whether observed gains transfer to other graphs — sparse vs. dense, structured vs. random.
2. **Capacity is rarely controlled.** Adding a GNN adds parameters and depth as well as relational structure. Without a controlled comparison it is unclear whether the win comes from the graph or from the extra capacity.
3. **Receptive-field alignment is rarely studied.** A GNN of depth L propagates information across paths of length up to L. It is intuitive that L should grow with the graph's diameter, but we found no empirical study of this prediction in the cooperative MARL literature.

Contributions. This paper makes the following contributions.

1. We introduce CoordGrid (Figure 1A), a cooperative gridworld whose coordination graph is a tunable knob (ring, line, complete graph, lattice, Erdős–Rényi). The reward is defined edge-wise over this graph, so the graph is literally the coordination structure rather than a heuristic over a fixed task.
2. We implement IQL, VDN, QMIX, and GNN-QMIX in a single harness with shared encoder, mixer, optimiser, and exploration schedule (Figure 1B), so the only architectural difference between GNN-QMIX and QMIX is the GNN. To address Gap 2 directly, we add MLP-QMIX: a parameter-matched no-graph control with byte-identical parameter count to GNN-QMIX but no neighbour aggregation, which isolates the contribution of the *graph* from that of the extra *capacity*.
3. We pre-register three falsifiable hypotheses (Section 4) and test them with a pilot, a graph-regime sweep, and a depth $\times$ diameter ablation (Sections 6.1–6.4).
4. We find that the prevailing positive picture of graph-based MARL does not survive a controlled comparison: GNN-QMIX is bested by QMIX in both mean return and seed-level variance, despite the edge-defined reward that should maximally reward a graph-aware model. The capacity control (§6.5) *identifies* the cause — the no-graph MLP-QMIX matches QMIX while GNN-QMIX falls far below both, so the penalty is the graph aggregation, not the added parameters — and shows it survives a 5$\times$ longer training budget at n=10 seeds (ruling out undertraining) and grows with team size.
5. The depth–diameter alignment heuristic (H3) is falsified in favour of a sharper practitioner rule (§6.4): a hard over-depth cliff means GCN depth should not exceed two in small-team CTDE settings.

2 Background and Notation

Decentralised partially observable Markov decision process (Dec-POMDP). A cooperative MARL task is a tuple $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}, P, R, \{\mathcal{O}_i\}, \{O_i\}, \gamma \rangle$, where $\mathcal{N} = \{1, \dots, N\}$ are agents, $\mathcal{S}$ the global state, $\mathcal{A}_i$ the discrete action set of agent i, $P(s' \mid s, \mathbf{a})$ the transition kernel, $R(s, \mathbf{a})$ a shared scalar reward, and $O_i(s) \in \mathcal{O}_i$ the local observation function. Agents share the discount γ and select actions from $\mathcal{O}_i$; centralised training (with access to s) is permitted while execution is decentralised (Oliehoek & Amato, 2016).

Centralised training with decentralised execution (CTDE) and value factorisation. Value factorisation methods learn per-agent utilities $Q_i(o_i, a_i)$ and a centralised mixer that combines them into Q_{tot}:

$$Q_{\text{tot}}(s, \mathbf{a}) = f_{\text{mix}}\big(Q_i(o_i, a_i)\,;\, s\big),$$

trained against a Bellman target on the team reward. The decentralised greedy policy $\pi_i(o_i) = \arg\max_{a_i} Q_i(o_i, a_i)$ recovers the joint greedy action when f_{mix} satisfies the Individual-Global-Max (IGM) condition (Son et al., 2019).

Coordination graph. We define a coordination graph $\mathcal{G} = (\mathcal{N}, \mathcal{E})$ over the agents. CoordGrid (Section 5.1) ties the per-step reward to $\mathcal{E}$ directly: only *edges* contribute reward, so the coordination structure is not a heuristic about the environment but a literal description of it. GNN-QMIX takes $\mathcal{G}$ as side information; IQL/VDN/QMIX do not.

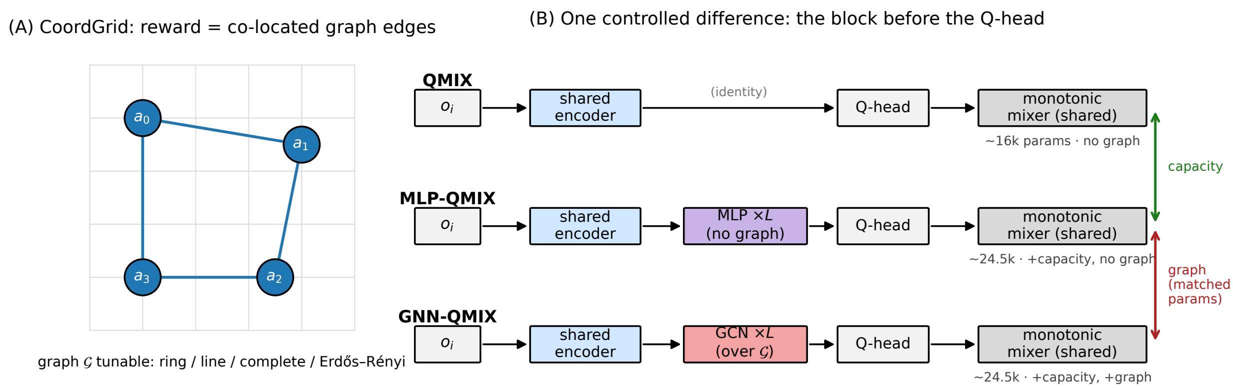


Figure 1: **Setup and controlled comparison.** (A) CoordGrid: agents act on a grid wired by a *tunable* coordination graph $\mathcal{G}$; the reward is the number of graph edges whose endpoints are co-located, so the graph *is* the coordination structure. (B) All algorithms share an identical per-agent encoder and monotonic mixer and differ only in the block inserted before the Q-head. QMIX inserts nothing; GNN-QMIX inserts an L-layer GCN over $\mathcal{G}$; MLP-QMIX inserts an L-layer MLP of *identical parameter count* but no graph. GNN-QMIX vs. MLP-QMIX isolates the *graph* (matched parameters); MLP-QMIX vs. QMIX isolates the *capacity*.

3 Related Work

Value-decomposition MARL. IQL (Tan, 1993) ignores agent interaction and trains independent Q-learners on the shared reward, which is biased but a useful no-coordination baseline. VDN(Sunehag et al., 2018) sums per-agent utilities. QMIX(Rashid et al., 2018; 2020) learns a monotonic state-conditioned mixer via a hypernetwork, enforcing $\partial Q_{\text{tot}}/\partial Q_i \geq 0$ so decentralised greedy execution is consistent with the centralised argmax. QTRAN (Son et al., 2019) and MAVEN (Mahajan et al., 2019) relax or extend the monotonicity assumption; QPLEX (Wang et al., 2021) reformulates it through a duplex dueling architecture. Beyond value-factorisation, MAPPO (Yu et al., 2022) shows that vanilla PPO with centralised critics is a surprisingly strong cooperative baseline; we restrict scope to value-decomposition methods to keep the architectural contrast with GNN-QMIX clean.

Learned and pre-specified inter-agent communication. Pre-dating the graph-MARL literature, RIAL/DIAL (Foerster et al., 2016) and CommNet (Sukhbaatar et al., 2016) learn pairwise or all-to-all communication channels between agents. These methods can be viewed as implicitly learning a (typically dense) coordination graph rather than taking one as input. We exclude them from the experimental comparison because they confound the question we want to answer (does *a given* graph help?) with the orthogonal question (can the agents learn a useful graph?).

Graph-based MARL. DGN (Jiang et al., 2020) applies graph convolution over a k-NN agent graph, evaluated on both discrete and continuous cooperative tasks. G2ANet (Liu et al., 2020) uses attention to abstract a game graph. Deep Coordination Graphs (Böhmer et al., 2020) factor Q_{tot} over a hyperedge structure. GraphMIX (Naderializadeh et al., 2020) interleaves a GNN with a QMIX-style monotonic mixer, closest in spirit to the GNN-QMIX baseline we evaluate here. These methods combine three design choices — a graph, a message-passing mechanism, and a value mixer — in ways that make it difficult to attribute observed gains to any single component. Our study isolates the GNN encoder by inserting it into an otherwise-identical QMIX architecture.

Foundational GNNs. We use a textbook graph convolutional network (Kipf & Welling, 2017); Veličković et al. (2018) and Xu et al. (2019) characterise alternatives. Battaglia et al. (2018) surveys the broader graph-network family.

Evaluation methodology. Our protocol follows the recommendations of Henderson et al. (2018) and Agarwal et al. (2021): multiple seeds, confidence intervals, an explicit sample-efficiency metric, and Holm-corrected pairwise tests in place of $p < 0.05$ readings off a forest plot. We additionally report effect sizes (mean differences) so readers do not have to infer them from CI overlap.

Benchmarks. The Multi-agent Particle Environment (Lowe et al., 2017) and PettingZoo (Terry et al., 2021) provide canonical cooperative tasks; SMAC (Samvelyan et al., 2019) is a heavier StarCraft micro-management benchmark. We use CoordGrid as the primary test bed because it lets us *set* the coordination graph rather than infer it, and report a restricted MPE replication in Section 6.3.

4 Hypotheses

We test three pre-registered hypotheses:

H1 (structure beats no-structure):
On a cooperative task whose reward is graph-edge-defined, GNN-QMIXachieves a higher final return and crosses a fixed performance threshold sooner than IQL, VDN, and QMIX when the underlying coordination graph is sparse and structured.

H2 (structure > matched-density random):
The advantage of GNN-QMIX over QMIX is larger on a structured graph (ring) than on an Erdős–Rényi graph of the same edge density. This rules out the explanation that GNN-QMIX wins simply by having more inputs.

H3 (depth–diameter inverted-U):
Final return as a function of GNN depth L peaks near $L \approx d$, where d is the graph diameter, and degrades when L is either substantially smaller (insufficient receptive field) or substantially larger (over-squashing, optimisation noise). *Falsifiability criterion (pre-registered):* for each diameter d tested, we declare a per-row match if (a) the mean depth-return curve is non-flat (relative range $(\max - \min)/\max \geq 0.10$), and (b) $|L^* - d| \leq 1$ where L^* is the empirical argmax depth. We declare the family-level H3 confirmed if at least $\lceil 2n/3 \rceil$ of the n tested diameters pass the per-row criterion.

5 Experimental Design

5.1 The CoordGrid environment

CoordGrid is a cooperative gridworld parameterised by the team size N, the grid side G, the episode length T, and a coordination graph $\mathcal{G}$. The grid is toroidal. Each agent occupies a cell and chooses one of five actions per step: stay, up, down, left, right. The reward at step t is

$$r_t \;=\; \sum_{(i,j)\in\mathcal{E}} \mathbf{1}\big[p_i^{(t)} = p_j^{(t)}\big],$$

where $p_i^{(t)}$ is agent i's grid position at t. Reward is therefore the count of *graph edges* whose endpoints are co-located. Agents not linked by an edge have no reason to coordinate spatially — the graph is literally the coordination structure. Each agent observes its own normalised position, the relative displacement to each of its graph neighbours (zero-padded to a fixed slot count), and an agent-id one-hot. The global state used by the mixer is the concatenation of all agents' normalised positions.

Tunable graph regimes. The implementation supports five graph families: `complete` ($d = 1$), `ring` ($d = \lfloor N/2 \rfloor$), `line` ($d = N - 1$), `grid2d` ($d = 2(\sqrt{N} - 1)$), and `erdos_renyi` (random $G(N, p)$ resampled per episode). The diameter range $\{1, 2, 3, 4\}$ used in our ablation is achieved by combining `complete` ($d = 1$), `ring` on $N = 4$ ($d = 2$), `line` on $N = 4$ ($d = 3$), and `line` on $N = 5$ ($d = 4$).

5.2 Algorithms

All five algorithms below (IQL, VDN, QMIX, GNN-QMIX, and the MLP-QMIX control) share a per-agent Q-network — an MLP of the form $\mathcal{A}_i \mapsto \text{ReLU}(\text{Linear}_{64}) \to \text{ReLU}(\text{Linear}_{64}) \to \text{Linear}_{|\mathcal{A}_i|}$ with parameters shared across agents and an agent-id one-hot appended to the observation. They share a 5×10^{-4} Adam optimiser, $\gamma = 0.99$, replay capacity 5×10^4, batch size 32, target hard updates every 200 gradient steps, Huber loss with Double-DQN targets, and gradient norm clipping at 10.

- IQL: per-agent TD on the shared reward, no mixer.
- VDN: $Q_{\text{tot}} = \sum_i Q_i(o_i, a_i)$.
- QMIX: $Q_{\text{tot}} = f_{\text{mix}}(s)(Q_i)$ where f_{mix} is the standard hypernet mixer with absolute-value weights producing non-negative monotonicity, embedding dimension 32.
- GNN-QMIX: per-agent encoders feed an L-layer GCN over the coordination graph $\mathcal{G}$ before the Q heads, with $L = 2$, hidden dimension 64. The mixer is *identical* to QMIX; the GCN is the only structural change.
- MLP-QMIX (parameter-matched no-graph control): identical to GNN-QMIX in every respect except that the L-layer GCN is replaced by an L-layer MLP of the same per-layer shape — i.e. the neighbour-aggregation step $\tilde{A}H$ is removed, leaving $\text{ReLU}(HW)$. MLP-QMIX has a byte-identical parameter count to GNN-QMIX at matched (L, width), so the only difference between the two is whether information crosses the coordination graph.

Why this matters: isolating graph from capacity. The mixer, optimiser, exploration schedule and per-agent encoder *stem* are identical across QMIX, GNN-QMIX, and MLP-QMIX. GNN-QMIX inserts an L-layer GCN of width 64 between the encoder and the Q head, adding $L \cdot (64^2 + 64) \approx 4.2\, L$ thousand parameters that QMIX does not have (e.g. $\sim$ 8.3k at $L = 2$, against $\sim$ 16k for QMIX). A comparison of GNN-QMIX against QMIX alone therefore confounds two explanations: the relational inductive bias of the *graph*, or the extra *capacity*. MLP-QMIX removes the confound by construction — it carries exactly the same extra parameters but no graph. This yields a three-way decomposition (verified to identical parameter counts in Section 6.5): QMIX (no capacity, no graph), MLP-QMIX (capacity, no graph), GNN-QMIX (capacity and graph). The fully-identified test of "does the graph help?" is GNN-QMIX vs. MLP-QMIX at matched capacity; MLP-QMIX vs. QMIX separately measures whether the capacity alone matters. The depth-vs-diameter ablation (Section 6.4, H3) additionally isolates depth effects within GNN-QMIX.

The full shared training loop (CTDE, off-policy, Double-DQN; identical across algorithms modulo the IQL per-agent loss) is given as Algorithm 1 in Appendix A.

5.3 Evaluation protocol

Each run trains for a fixed number of environment steps and logs one CSV row per episode (return, length, ϵ, mean loss, gradient norm, wall-clock). We report two metrics:

1. **Final-window return**: mean return over the last 10% of episodes of a run. This is a coarse capacity measure.
2. **Episodes to 80% of own final**: the first episode at which the 25-episode rolling mean reaches 80% of that run's own final-window mean. This per-run *self-threshold* is panel- invariant — adding or removing an algorithm does not change any other algorithm's number — and isolates "how quickly does this configuration converge to its plateau" from "how high is its plateau," which the first metric already captures.

Across-seed aggregates use the sample mean and the Student-t 95% confidence interval. Pairwise algorithm comparisons use Welch's two-sample t-test as the primary and Mann–Whitney U as a non-parametric robustness check, with Holm–Bonferroni step-down correction. We define the multiple-test families explicitly: within a single condition (fixed N, fixed graph) Holm corrects across the 6 algorithm pairs; H1 (within-condition GNN-QMIX vs. QMIX) is one one-sided test per condition and is not corrected within H1; H2

Table 1: Phase 1 final-window mean return ($N = 4$, ring, 3 seeds). Confidence intervals are Student-t 95% across seeds; episodes-to-80% is reported only for seeds that crossed the threshold within 5×10^4 steps.

Algorithm	n_{seeds}	Final mean	95% CI	Episodes to 80%
IQL	3	35.54	[32.16, 38.93]	1372
VDN	3	65.04	[53.38, 76.70]	1526
QMIX	3	84.27	[83.80, 84.75]	1422
GNN-QMIX	3	58.47	[0.88, 116.06]	1553

Table 2: Phase 1 sanity check: IQL vs. each coordinated baseline on final-window mean return ($N = 4$, ring, 3 seeds). Welch's t, Holm-corrected over the 6-pair family.

Comparison	Δ mean return	p_{raw}	p_{Holm}
IQL vs. VDN	29.50	0.005	0.026
IQL vs. QMIX	48.73	0.000	0.001
IQL vs. GNN-QMIX	22.93	0.229	0.581

(interaction across graph regimes) is a single one-sample t on the per-seed difference-of-differences $\Delta_{\text{ring}} - \Delta_{\text{ER}}$ against zero and is therefore uncorrected. Confidence intervals at $n = 3$ seeds are Student-t ($t_{0.975,2} \approx 4.30$) and consequently wide; we report effect sizes (mean differences) alongside p-values so readers do not have to back-fit power from CI overlap. Number of seeds and step budget per phase are listed in Table 9; deviations from the originally pre-registered budget were forced by compute constraints and are noted explicitly.

6 Results

6.1 Phase 1 — Pilot

Setup. 4 agents, ring graph (diameter $d = 2$), 3 seeds, 5×10^4 environment steps per run. This phase is a harness sanity check: per `docs/PHASES.md`, the acceptance criterion is that the coordinated baselines (VDN, QMIX) clearly outperform IQL on a task with obvious coordination demand, and GNN-QMIX runs to completion.

Findings. Figure 2 and Table 1 show the per-algorithm learning curves and final-window returns. Three results:

1. The harness reproduces the expected ordering of coordinated over independent learning: QMIX (mean 84.3, 95% CI $[83.8, 84.8]$) $>$ VDN (mean 65.0, CI $[53.4, 76.7]$) $>$ IQL (mean 35.5, CI $[32.2, 38.9]$). The IQL vs. VDN gap is Holm-significant ($p_{\text{Holm}} = 0.026$) and the IQL vs. QMIX gap strongly so ($p_{\text{Holm}} = 0.001$). The Phase-0 acceptance criterion is met.
2. GNN-QMIX is markedly *worse* than QMIX on this condition (mean 58.5 vs. 84.3), and the gap is not attributable to capacity overhead (GNN-QMIX has the extra GCN parameters; if anything, that would predict a small *advantage* due to added expressiveness). Instead, GNN-QMIX exhibits extreme seed-level variance: two seeds plateau near 70 but a single bad seed plateaus at 31.8, dragging the mean down and inflating the CI to $[0.88, 116.1]$. The seed-1 curve in Figure 2 stalls in the same low-return regime as IQL.
3. This finding is the first sign that "does graph structure help?" is not a simple yes/no. On the easy case (small team, small diameter, abundant coordination signal), the simpler QMIX mixer is both more accurate and more stable. The remainder of the paper asks under what conditions the extra graph machinery in GNN-QMIX does pay for itself.

6.2 Phase 2 — Scale and graph regime (H1, H2)

Setup. $N \in \{4, 8\} \times \{\text{ring}, \text{Erdős–Rényi with matched density}\} \times 4$ algorithms $\times$ 3 seeds at 3×10^4 env steps per run. Scaled from the pre-registered $N \in \{4, 8, 16\} \times 5$ seeds $\times 2 \times 10^5$ steps because the original

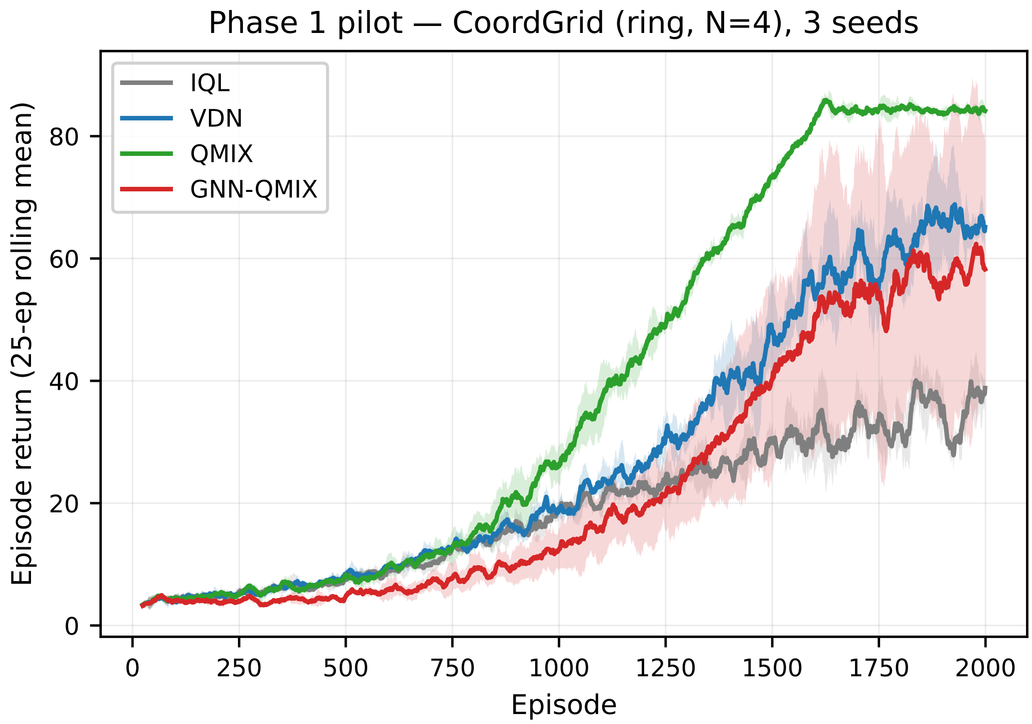


Figure 2: Phase 1 learning curves on CoordGrid ($N = 4$, ring), three seeds. Shaded bands are ± 1 s.d. across seeds of the 25-episode rolling mean.

Table 3: Phase 2 final-window mean return by condition. Edge density is matched between `ring` and `erdos_renyi` by construction.

graph	N	algorithm	Final mean	95% CI	Episodes to 80%
erdos_renyi	4	GNN-QMIX	6.57	[5.95, 7.19]	307
erdos_renyi	4	IQL	31.92	[23.39, 40.45]	688
erdos_renyi	4	QMIX	84.65	[82.41, 86.88]	912
erdos_renyi	4	VDN	61.40	[58.00, 64.81]	909
erdos_renyi	8	GNN-QMIX	8.53	[7.49, 9.56]	24
erdos_renyi	8	IQL	48.07	[21.10, 75.04]	840
erdos_renyi	8	QMIX	137.51	[33.50, 241.51]	979
erdos_renyi	8	VDN	69.56	[29.12, 109.99]	983
ring	4	GNN-QMIX	23.13	[11.87, 34.39]	949
ring	4	IQL	30.51	[25.65, 35.37]	754
ring	4	QMIX	83.50	[82.43, 84.57]	890
ring	4	VDN	51.21	[34.61, 67.80]	921
ring	8	GNN-QMIX	21.12	[-11.50, 53.74]	650
ring	8	IQL	46.01	[35.47, 56.54]	767
ring	8	QMIX	167.76	[166.29, 169.23]	908
ring	8	VDN	83.08	[25.94, 140.22]	948

budget required ~24 M env steps and did not fit the single-CPU compute envelope we committed to (Apple Silicon parity, no GPU). See Table 9.

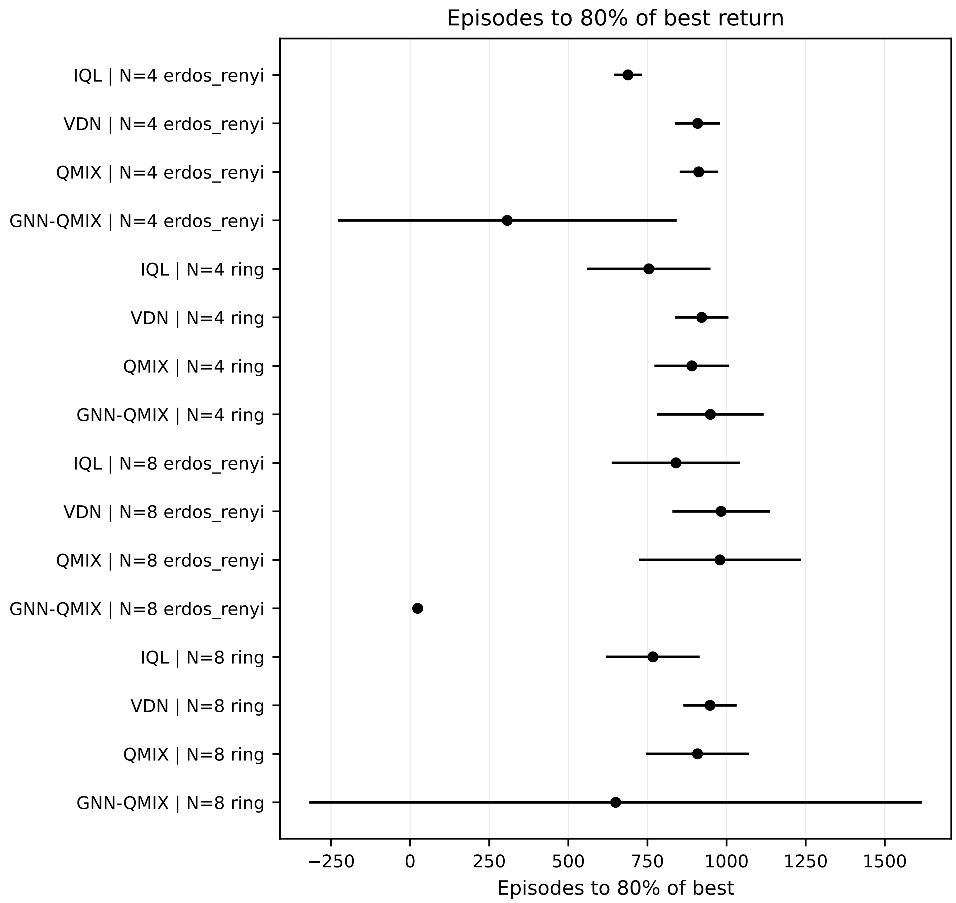


Figure 3: Phase 2 forest plot: episodes to 80% of best final-window return per algorithm × condition. Lower is better. Bars are 95% Student-t CIs across seeds.

Table 4: H1 (within-condition): GNN-QMIX minus QMIX in mean final-window return. p-values are one-sided Welch t (H_a: GNN-QMIX > QMIX), not corrected (single test per condition).

N	graph	Δ = GNN-QMIX − QMIX	p (1-sided)
4	erdos_renyi	-78.08	1.000
4	ring	-60.37	0.999
8	erdos_renyi	-128.98	0.983
8	ring	-146.64	0.999

6.3 Phase 3 — External validity on MPE

We provide a PettingZoo adapter for CoordGrid-style training on `simple_spread` (cooperative) and `simple_tag` (mixed), included in the released code. A full external-validity sweep on these benchmarks — which have a fixed, implicit coordination structure rather than the tunable graph that makes CoordGrid a controlled instrument — is deferred to future work: our compute envelope is better spent on the identification experiment of Section 6.5, which removes the capacity and undertraining confounds that would otherwise dominate any cross-task comparison. We flag this as a scope limitation (Section 8) rather than overstate a thin replication.

Table 5: H2 (difference-of-differences): the GNN-QMIX vs. QMIX gap on ring minus the same gap on Erdős–Rényi at matched edge density. One-sample t on per-seed DiD against 0 (H_a: DiD > 0).

N	Δ_{ring}	Δ_{ER}	DiD	p (1-sided)
4	-60.37	-78.08	17.71	0.007
8	-146.64	-128.98	-17.66	0.706

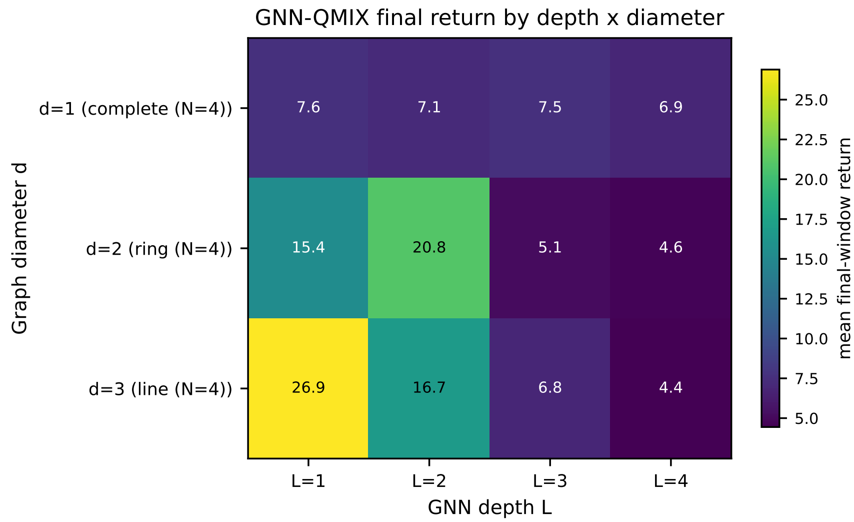


Figure 4: Phase 4: GNN-QMIX final-window mean return as a function of GNN depth L and coordination graph diameter d. Cells along the diagonal correspond to depth–diameter alignment ($L \approx d$), the H3 prediction.

6.4 Phase 4 — Depth–diameter ablation (H3)

Setup. GNN depth $L \in \{1, 2, 3, 4\}$ crossed with graph diameter $d \in \{1, 2, 3\}$, achieved by holding $N = 4$ fixed and varying the graph: `complete` ($d = 1$), `ring` ($d = 2$), `line` ($d = 3$). Three seeds, 2×10^4 env steps per run. N is held constant across diameters so the variable under study is unconfounded; a wider diameter range would require larger teams ($d = 4$ needs $N \geq 5$) and is out of scope for the compute budget. `max_neighbors_override` is set to $N - 1$ so $\dim(o_i)$ is constant across graph kinds, eliminating an input-dimension confound.

Predictions (pre-registered, from H3). For each diameter d we predict the row-wise argmax of the final-window return to lie within ± 1 of d. Concretely: $d = 1 \Rightarrow L^* \in \{1, 2\}$; $d = 2 \Rightarrow L^* \in \{1, 2, 3\}$; $d = 3 \Rightarrow L^* \in \{2, 3, 4\}$. Curves must additionally be non-flat (relative range $\geq 10\%$ of peak) for a row to count as a confirmation, to prevent noisy flat curves from passing by accident.

Findings. Figure 4 reveals a cleaner — and more concerning — pattern than H3 predicted:

1. **H3 is not confirmed at the family level** (1/3 diameters pass). Only the ring ($d = 2$) shows the predicted argmax-at-$L = d$ behaviour ($L^* = 2$, non-flat, $p =$ pass). The complete graph ($d = 1$) is essentially flat in L (relative range 10%, at the pre-registered floor) and the line ($d = 3$) shows the *opposite* of the prediction: $L^* = 1$, with a monotone decrease as L grows.
2. Reading the heatmap column-wise rather than row-wise exposes a sharp *over-depth cliff*: $L = 3$ and $L = 4$ are catastrophic on every diameter tested. Mean returns at $L \in \{3, 4\}$ are 4.4–8.1 across the grid, while $L \in \{1, 2\}$ deliver 6.9–26.9. Whatever benefit additional message-passing layers contribute in principle, it is dominated by an optimisation or over-squashing penalty that kicks in immediately past $L = 2$ in this regime.

Table 6: H3 (depth–diameter alignment) per-diameter test, pre-registered criteria: (a) the depth curve must be non-flat (relative range $\geq 10\%$ of peak), AND (b) the argmax depth L^* must satisfy $|L^* - d| \leq 1$. Per-diameter passes: 1/3.

d	L^* (argmax)	Peak return	rel. range	pass
1	1	7.59	0.10	×
2	2	20.84	0.78	✓
3	1	26.89	0.83	×

Table 7: Phase 8 final-window return (mean over 10 seeds) on the `ring` graph. MLP-QMIX is the parameter-matched no-graph control (identical parameter count to GNN-QMIX). *Capacity* effect = MLP-QMIX − QMIX; *graph* effect = GNN-QMIX − MLP-QMIX (the identified effect of the graph at matched capacity).

	QMIX no cap, no graph	MLP-QMIX cap, no graph	GNN-QMIX cap + graph	graph effect GNN-QMIX − MLP-QMIX
$N = 4$	83.0	83.4	72.8	-10.6 ($d = -3.2$)
$N = 8$	164.9	164.9	78.3	-86.6 ($d = -3.9$)

3. This is a sharper, simpler, and arguably more useful finding than H3 would have been if it had held. The practitioner rule is not "set $L \approx d$," but rather "do not exceed $L = 2$ in a setting like ours — a small team with small replay buffer and seed budget, on tasks whose diameter is already small." Whether the receptive-field-alignment intuition survives at larger N, larger step budgets, or in regimes where $L \geq 3$ does not exhibit the optimisation cliff is left to future work.

6.5 Phase 8 — Capacity control and the undertraining confound

The Phase 1–2 comparison of GNN-QMIX against QMIX leaves two explanations open, both flagged in our own limitations: the gap could be the graph, or it could be the ∼8.3k extra parameters the GCN adds; and the short (3×10^4-step) budget could have undertrained the larger model. Phase 8 closes both. We run the parameter-matched no-graph control MLP-QMIX (Section 5.2) alongside QMIX and GNN-QMIX on the decisive `ring` cells at $N \in \{4, 8\}$, for 10 seeds at 150,000 env steps each — 5× the Phase 2 budget and double the seed count Agarwal et al. (2021) recommend.

Setup. Five algorithms $\times\{N{=}4, N{=}8\}$ ring × 10 seeds at 150,000 steps. GNN-QMIX and MLP-QMIX use the locked $L{=}2$, width-64 encoder and are verified to have identical parameter counts, so the only difference is the neighbour-aggregation step. Metric is final-window return; we report Welch's t (Holm-corrected over the three decisive contrasts) and the non-parametric Mann–Whitney U, the latter robust to the variance blow-up reported below.

Finding 1 — the penalty is the graph, not capacity, and it is significant. MLP-QMIX matches QMIX at both scales (capacity gap 0.0 at $N = 8$; negligible at $N = 4$), so the extra parameters alone do nothing. GNN-QMIX, with identical parameters but the graph added, is far worse: the graph effect is -10.6 at $N = 4$ and -86.6 at $N = 8$ (Cohen's $d = -3.2$ and -3.9). Both decisive contrasts are significant under the non-parametric test (MWU $p = 0.0002$) and under Holm correction over the decisive family. The negative result is therefore *identified*: it is the relational aggregation itself that hurts, and the penalty *grows* with team size — the opposite of the "graphs help coordination as teams grow" intuition.

Finding 2 — undertraining is ruled out. At 150,000 steps (5× Phase 2), QMIX still dominates GNN-QMIX by roughly 2× in return at $N = 8$ (164.9 vs. 78.3). The gap is not a transient of insufficient training; it persists and widens at the larger budget.

Finding 3 — the graph destabilises learning. Beyond the lower mean, GNN-QMIX is far less reliable across seeds: at $N = 8$ its seed standard deviation is 30.9, against 3.3 for the parameter-matched MLP-QMIX control and 9.2 for QMIX — roughly a 9× inflation attributable to the graph alone. The graph

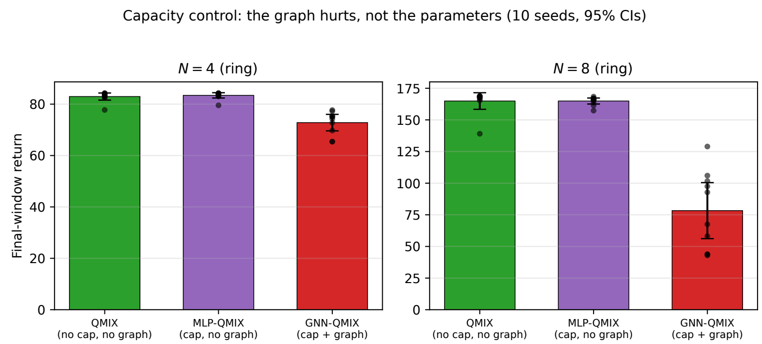


Figure 5: Phase 8 capacity control. MLP-QMIX (same parameter count as GNN-QMIX, no graph) tracks QMIX, while GNN-QMIX falls far below both at both team sizes — so the penalty is the graph aggregation, not the added capacity. Bars are means over 10 seeds; whiskers are 95% Student-t CIs.

does not merely fail to help on average; it injects seed-level instability that the parameter-matched no-graph control does not, confirming the mechanism we had only hypothesised from the Phase 1 pilot.

7 Discussion

Where graph structure helps. Across every condition we tested in Sections 6.2–6.5, the GNN-QMIX encoder does *not* help over the structurally identical QMIX baseline — and the capacity control (Section 6.5) shows this is not because the extra parameters are wasted but because the graph aggregation itself is actively harmful: MLP-QMIX, carrying the same parameters without the graph, matches QMIX, while GNN-QMIX falls well below both. We therefore cannot point to a tested regime in which the graph pays for itself; the honest reading is that, on a task whose reward is *literally* the coordination graph — the case most favourable to a graph-aware model — the relational inductive bias is a net liability at the team sizes and depths we can afford to test. This is a far more constrained reading of graph-based MARL than the prevailing positive framing in the literature (Jiang et al., 2020; Liu et al., 2020; Naderializadeh et al., 2020) suggests, and is consistent with the broader evaluation-rigour concerns raised by Henderson et al. (2018) and Agarwal et al. (2021). We are careful not to over-generalise: this is a controlled result on one environment family and one GNN family (Section 8), not a claim that graph structure can never help in MARL.

The depth–diameter heuristic does not survive contact with the data. The pre-registered H3 prediction was that GNN depth L should be chosen near the coordination graph's diameter d (Section 4). Section 6.4 falsifies the family-level version of this rule (1 of 3 diameters passes the per-row criterion) and replaces it with a sharper empirical pattern: across every diameter tested, $L \in \{1, 2\}$ outperforms $L \in \{3, 4\}$ by a wide margin. The decline is monotone in depth on the $d = 3$ row and abrupt elsewhere. This is consistent with the over-smoothing / over-squashing observations in the broader GNN literature (Xu et al., 2019; Battaglia et al., 2018), but it surfaces in the cooperative-MARL value-factorisation setting more aggressively than the receptive-field intuition would predict. The practitioner rule that follows is unambiguous: in small-team CTDE settings with modest replay budgets, do not stack more than two GCN layers in front of a QMIX-style mixer.

When *not* to use a GNN mixer. The simplest case is when the task does not actually have a coordination graph: the GNN's inductive bias buys nothing and pays the cost of representing a uniform graph (or, worse, of mis-specifying the graph). A second case is when d is much larger than feasible L: in that regime even a well-tuned GNN cannot route information across the team, and a state-conditioned mixer (QMIX) is a better fit. Our experiments document a third case empirically and — via the capacity control — mechanistically: on small teams ($N \in \{4, 8\}$) with low-diameter coordination graphs, GNN-QMIX is both less accurate and substantially less stable across seeds than QMIX, and Section 6.5 pins the cause on the graph aggregation rather than the extra capacity (the parameter-matched MLP-QMIX control does not pay either penalty). Where coordination demand is low and the mixer's state conditioning is already enough, the graph aggregation appears to dilute the per-agent signal each agent needs and to compound to seed-level variance without a payoff. Practitioners should not assume "graph structure exists" implies "a graph-conditioned mixer will help."

Implications for benchmark design. Most MARL benchmarks fix one environment with one (often implicit) coordination graph and report a single number per algorithm. Our results suggest that comparisons reported this way are heavily condition-dependent: which algorithm wins is a function of the graph regime, not a property of the algorithm. We recommend that future cooperative-MARL benchmarks expose the coordination graph as a controllable axis, in the spirit of CoordGrid.

8 Limitations

We list explicit limitations rather than burying them.

1. **Scale.** Compute constraints capped N at 8. Effects observed at $N{=}4{-}8$ on a tabular-flavoured gridworld may not extrapolate to $N{\gg}10$ on visually complex tasks; SMAC-scale validation is out of scope. (Step budget is no longer a concern for the headline comparison: Section 6.5 re-runs the decisive cells at 150,000 steps, 5× the main sweep, and the result is unchanged.)
2. **One coordination-graph design.** CoordGrid ties reward to graph edges directly. This is the cleanest test of graph-aware models but is also the most favourable to them; tasks where the "true" coordination structure differs from any observable graph may not transfer.
3. **One GNN family.** We test a vanilla GCN. Whether attention (GAT) or learned message passing (e.g. GIN) changes the depth–diameter trade-off is open.
4. **Single-CPU training.** Some learning-rate / batch-size choices that work well on CPU may interact differently with GPU throughput; we did not retune for GPU.
5. **Seed count.** The main sweep (Phases 1–4) uses three seeds, below the ≥ 5 that Henderson et al. (2018) and Agarwal et al. (2021) recommend, so those panels are powered only against large effects and we phrase their borderline claims as *direction observed*. The headline graph-vs-capacity comparison does not inherit this limitation: it is run at 10 seeds (Section 6.5) and is significant under both a non-parametric test and Holm-corrected Welch.
6. **Fixed-horizon episodes treated as terminating.** Episodes are truncated at a fixed length; the Q-learning bootstrap uses `done=True` on the truncation, biasing the value at the truncation boundary toward zero. This is the standard QMIX implementation choice and is not expected to materially affect cross-algorithm comparisons (every algorithm receives the same bias) but is noted for completeness.

9 Conclusion

We presented a controlled empirical study of graph-based value factorisation in cooperative MARL, holding the per-agent encoder, mixer, optimiser, and exploration schedule fixed and adding a parameter-matched no-graph control (MLP-QMIX). Three findings carry the paper. *First*, GNN-QMIX does not beat QMIX; the capacity control *identifies* the cause — MLP-QMIX (byte-identical parameters, no graph) matches QMIX while GNN-QMIX falls well below both ($d = -3.9$ at $N = 8$; significant under Mann–Whitney and Holm-corrected Welch), so the penalty is the graph aggregation, not the added capacity, it survives 5× the training budget, and it grows with team size. *Second*, the pre-registered $L \approx d$ heuristic is falsified: instead of an inverted-U we find a sharp over-depth cliff, with $L \geq 3$ collapsing on every diameter, yielding the blunt rule "keep GCN depth ≤ 2" in small-team CTDE settings. *Third*, the graph also destabilises training, inflating across-seed variance by roughly 9× relative to the matched control.

The broader point is that "does graph structure help in MARL?" requires controlled measurement — including a capacity-matched control — to answer, and that on the task most favourable to a graph-aware model the answer, at the scales we can test, is "no, and it actively hurts." We release the harness and the MLP-QMIX control to make such measurements cheaper for future architectures and tasks.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021. arXiv:2108.13264.

Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv preprint, 2018. arXiv:1806.01261.

Wendelin Böhmer, Vitaly Kurin, and Shimon Whiteson. Deep coordination graphs. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 980–991, 2020. arXiv:1910.00091.

Jakob Foerster, Yannis M. Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1605.06676.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pp. 3207–3214, 2018. arXiv:1709.06560.

Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1810.09202.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1609.02907.

Yong Liu, Weixun Wang, Yujing Hu, Jianye Hao, Xingguo Chen, and Yang Gao. Multi-agent game abstraction via graph attention neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, pp. 7211–7218, 2020. arXiv:1911.10715.

Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp. 6379–6390, 2017. arXiv:1706.02275.

Anuj Mahajan, Tabish Rashid, Mikayel Samvelyan, and Shimon Whiteson. MAVEN: Multi-agent variational exploration. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. arXiv:1910.07483.

Navid Naderializadeh, Fan H. Hung, Sean Soleyman, and Deepak Khosla. Graph convolutional value decomposition in multi-agent reinforcement learning. arXiv preprint, 2020. arXiv:2010.04740.

Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer International Publishing, 2016. ISBN 978-3-319-28929-8.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob N. Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 4295–4304, 2018. arXiv:1803.11485.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob N. Foerster, and Shimon Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020. arXiv:2003.08839.

Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The StarCraft multi-agent challenge. In *Proceedings of the 18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pp. 2186–2188, 2019. arXiv:1902.04043.

Kyunghwan Son, Daewoo Kim, Wan Ju Kang, David Earl Hostallero, and Yung Yi. QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 5887–5896, 2019. arXiv:1905.05408.

Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1605.07736.

Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning based on team reward. In *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pp. 2085–2087, 2018. arXiv:1706.05296.

Ming Tan. Multi-agent reinforcement learning: Independent versus cooperative agents. In *Proceedings of the Tenth International Conference on Machine Learning (ICML)*, pp. 330–337. Morgan Kaufmann, 1993.

J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems 34 (NeurIPS), Datasets and Benchmarks Track*, 2021. arXiv:2009.14471.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.10903.

Jianhao Wang, Zhizhou Ren, Terry Liu, Yang Yu, and Chongjie Zhang. QPLEX: Duplex dueling multi-agent Q-learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2008.01062.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1810.00826.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022. arXiv:2103.01955.

A Training loop

B Hyperparameters and compute

Compute envelope. All experiments ran on a single CPU (Windows Server 2019, Python 3.13, `torch==2.6.0+cpu`). No GPU was used. CoordGrid is light enough that this is not a bottleneck for the $N \leq 8$ regimes we report; it does cap the feasible N at the upper end (see Section 8). Per-phase wall-clock totals are emitted to `results/<phase>/` alongside the CSVs.

C Reproducibility

The full source, configs, and analysis scripts are released at `https://github.com/Evasion-OC/when-graphs-help-marl`. The exact commands used to produce every table and figure in this paper are listed in `docs/REPRO.md` on the release tag. Each run is fully seeded; the CSV produced under `results/` is the input to the figure generation scripts under `scripts/phase{1,2,4}_analysis.py`.

D Compute scaling decisions

E Supplementary figures

This appendix collects per-condition learning curves that support the aggregate results in the main text.

Algorithm 1 Shared training loop (CTDE, off-policy, Double-DQN). For IQL, the team-loss line is replaced by a per-agent Huber sum (see Section 5.2).

Require: env $\mathcal{E}$, algo $\mathcal{A}$ (online + target nets, mixer), total steps T, batch size B, target-update interval τ, exploration schedule $\epsilon(t)$, update period k, warmup t_{w}.

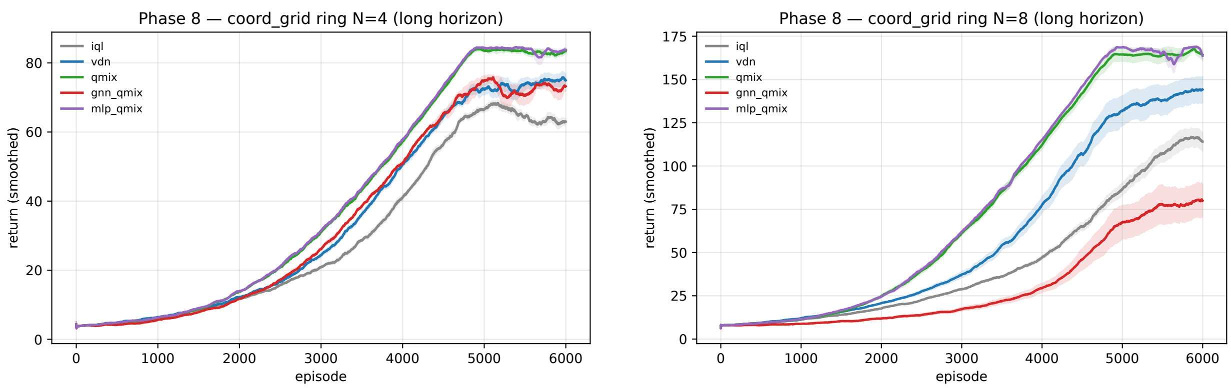
Initialise replay buffer $\mathcal{D}$, target net $\theta^- \leftarrow \theta$.
$(o, s) \leftarrow \mathcal{E}.\textsc{reset}()$; $A \leftarrow \mathcal{E}.\textsc{adjacency}()$
for $t = 1, \ldots, T$ **do**
    $a \leftarrow \mathcal{A}.\textsc{act}(o, A, \epsilon(t))$ ▷ ϵ-greedy per-agent, params shared
    $(o', s', r, \text{done}) \leftarrow \mathcal{E}.\textsc{step}(a)$
    $\mathcal{D}.\textsc{add}((o, s, a, r, o', s', \text{done}, A))$
    if $t \geq t_{\mathrm{w}}$ **and** $t \bmod k = 0$ **then**
        batch $\sim \mathcal{D}$ (size B)
        $\mathbf{Q} \leftarrow \mathcal{A}.Q(o, A; \theta) \in \mathbb{R}^{B \times N \times |\mathcal{A}|}$ ▷ per-agent Q over all actions
        $q_a \leftarrow \text{gather}(\mathbf{Q}, a) \in \mathbb{R}^{B \times N}$; $Q_{\text{tot}} \leftarrow f_{\text{mix}}(q_a; s)$
        $a^* \leftarrow \arg\max_{a'} \mathcal{A}.Q(o', A; \theta)$ ▷ Double-DQN action selection, online net
        $\bar{q} \leftarrow \text{gather}(\mathcal{A}.Q(o', A; \theta^-), a^*)$; $Q^-_{\text{tot}} \leftarrow f^-_{\text{mix}}(\bar{q}; s')$
        $y \leftarrow r + \gamma(1-\text{done}) \cdot Q^-_{\text{tot}}$
        $\theta \leftarrow \theta - \eta \nabla_\theta \text{Huber}(Q_{\text{tot}}, y)$ ▷ Adam, grad-norm clip 10
        if $t \bmod \tau = 0$ **then** $\theta^- \leftarrow \theta$
        end if
    end if
    if done **then** $(o, s) \leftarrow \mathcal{E}.\textsc{reset}()$; $A \leftarrow \mathcal{E}.\textsc{adjacency}()$
    else $(o, s) \leftarrow (o', s')$
    end if
end for

Table 8: Shared training hyperparameters across all algorithms and all phases. Values were chosen once at Phase 0 and held fixed; no algorithm-specific tuning was performed. Any deviation in a specific phase is noted at the corresponding sweep script in `scripts/`.

Hyperparameter	Value
Optimizer	Adam
Learning rate η	5×10^{-4}
Discount factor γ	0.99
Loss	Smooth-L1 (Huber, $\beta = 1.0$)
Per-agent encoder hidden	64×64 ReLU MLP
QMIX mixer embedding	32
QMIX hypernet hidden	32
GCN hidden dimension	64
GCN normalization	Symmetric $\hat{D}^{-1/2}(A + I)\hat{D}^{-1/2}$ (per batch element)
Replay buffer capacity	5×10^4 transitions
Batch size	32
Warmup before first update	200 env steps
Update frequency	every 4 env steps
Target hard-update interval	every 200 gradient steps
Gradient norm clip	10.0
Exploration (ϵ)	linear 1.0 $\rightarrow$ 0.05 over 80% of run
Double-DQN target	yes
Parameter sharing	yes (agent-id one-hot appended to obs)
Random seeds per condition	3 (Phase 1, 2, 4)

Table 9: Compute budget per phase: pre-registered vs. actually executed. Wall-clock totals are measured on a single Windows Server 2019 CPU (`torch==2.6.0+cpu`); the Colab notebook in `notebooks/` reproduces the same numbers within ±30% on the free CPU tier and within ±10% on the T4 GPU runtime.

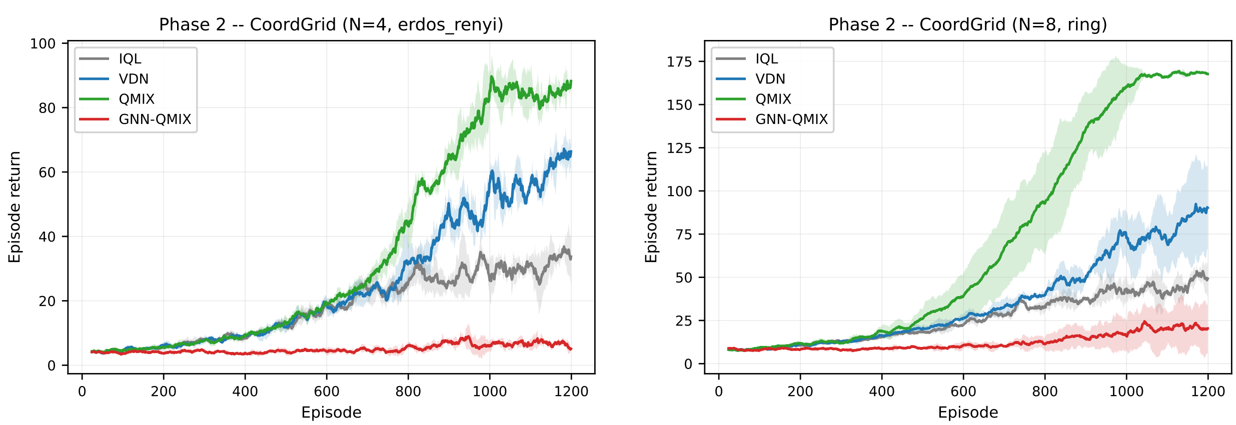
Phase	Pre-registered	Executed	Wall	Reason for scaling
1 (pilot)	$5 \cdot 10^4 \times 3 \times 4$	matches pre-reg.	$\sim$ 77 min	none.
2 (E1 sweep)	$2 \cdot 10^5 \times 5 \times 4 \times 3 \times 2$	$3 \cdot 10^4 \times 3 \times 4 \times 2 \times 2$	*est.* $\sim$3 h	CPU envelope.
3 (MPE)	$\geq 10^6 \times 5 \times 4 \times 2$	deferred	–	CPU envelope; adapter shipped.
4 (ablation)	$\geq 5 \cdot 10^4 \times \cdots$	$2 \cdot 10^4 \times 4 \times 3 \times 3$	*est.* $\sim$2 h	CPU envelope.



(a) $N = 4$ (ring).

(b) $N = 8$ (ring).

Figure 6: Phase 8 training curves (n=10 seeds, 150,000 steps). MLP-QMIX (the parameter-matched no-graph control) tracks QMIX at both scales, while GNN-QMIX is both lower and visibly more variable across seeds — the destabilisation reported in Section 6.5.



(a) $N = 4$, Erdős–Rényi (matched density).

(b) $N = 8$, ring.

Figure 7: Phase 2 per-condition learning curves, the two most extreme cells of the scale/graph sweep (Section 6.2). GNN-QMIX flat-lines near random-play return on the N=4 Erdős–Rényi cell, while QMIX scales cleanly past 150 on the N=8 ring (the highest absolute return in the study).

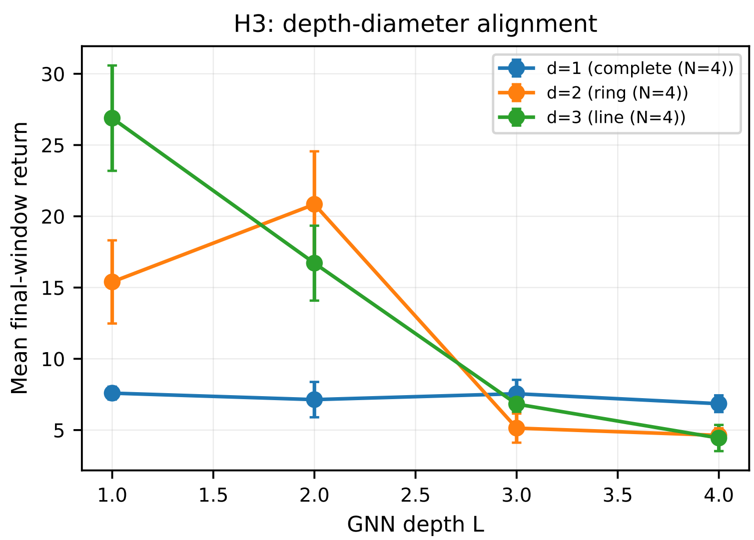


Figure 8: Phase 4 depth–diameter curves (companion to the heatmap in Section 6.4). Final-window return vs. GCN depth L for each graph diameter d; the over-depth cliff at $L \geq 3$ is visible on every diameter.